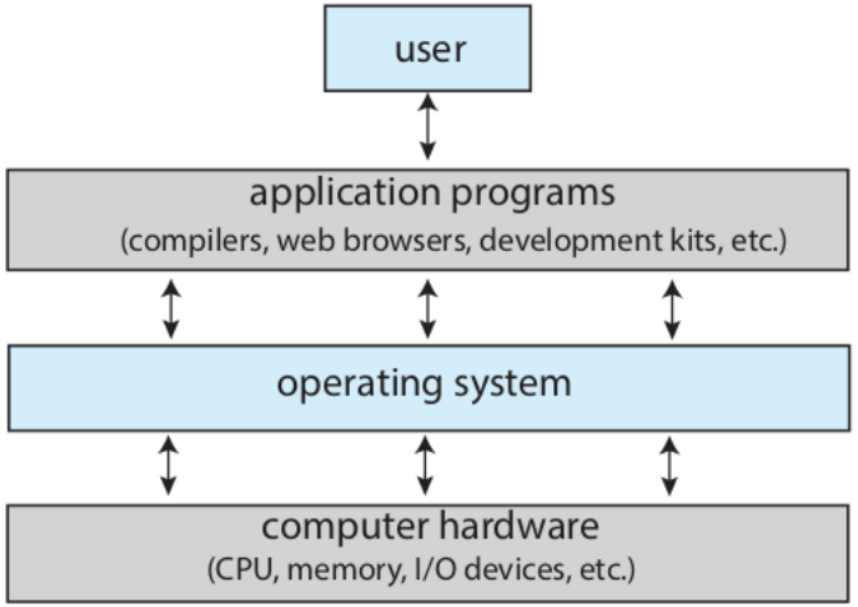
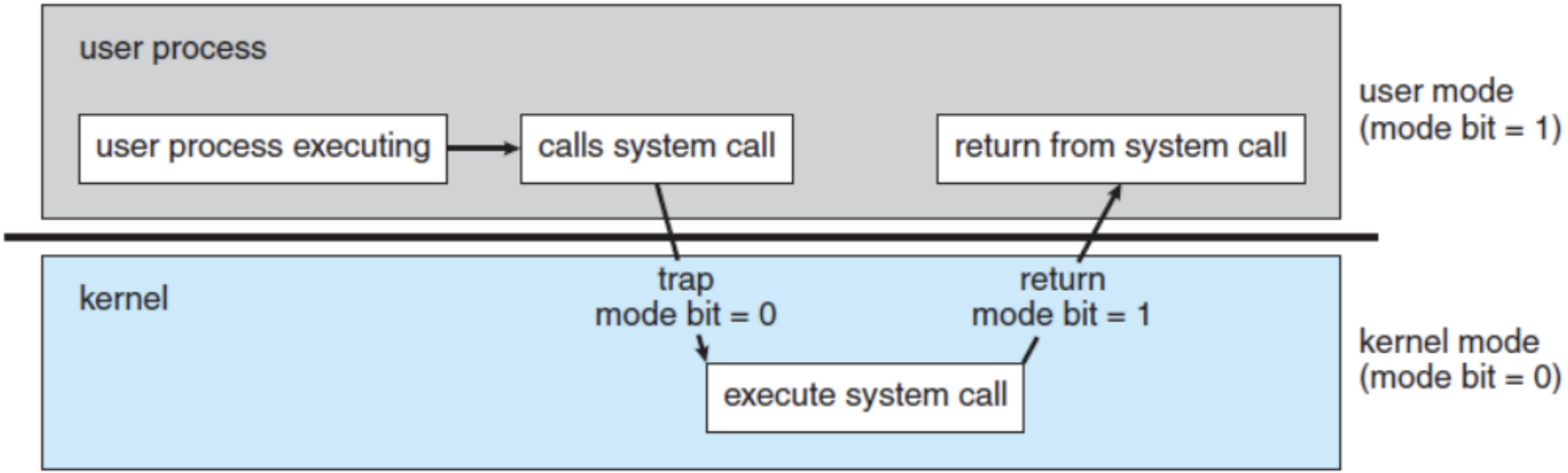
Recap: what is OS

- OS is an intermediary between a user of a computer and the computer hardware.
  - OS is a resource allocator.
    - Manages all resources: CPU, memory, disk, network, …
    - Decides between conflicting requests for efficient and fair resource use.
  - OS is a controller
    - Controls execution of programs to prevent errors and improper use of the computer.



Recap: Dual-Mode Operation

- The dual-mode operation allows the OS to protect itself and other system components.
- User mode and kernel mode.
- The system call changes to the kernel mode, and then return from the call resets it to the user.



Recap: Interrupt, Trap, Fault

How does the kernel get the CPU back after a user program holds the CPU?

Example	Disk finished; timer fired; key pressed	System Call	Page fault; divide-by-zero error
Mechanisms	**Interrupt**	**Trap**	**Fault**
Timing	Asynchronous	Synchronous	Synchronous

Example 1: interrupt

event occurs (e.g., I/O event finishes) → CPU finishes the current instruction → index the interrupt vector table → switch to kernel mode → run the registered handler → restore state, return to user mode.

Example 2: system call

syscall is a CPU instruction. When done, a return-from-syscall instruction switches back to user mode



Process

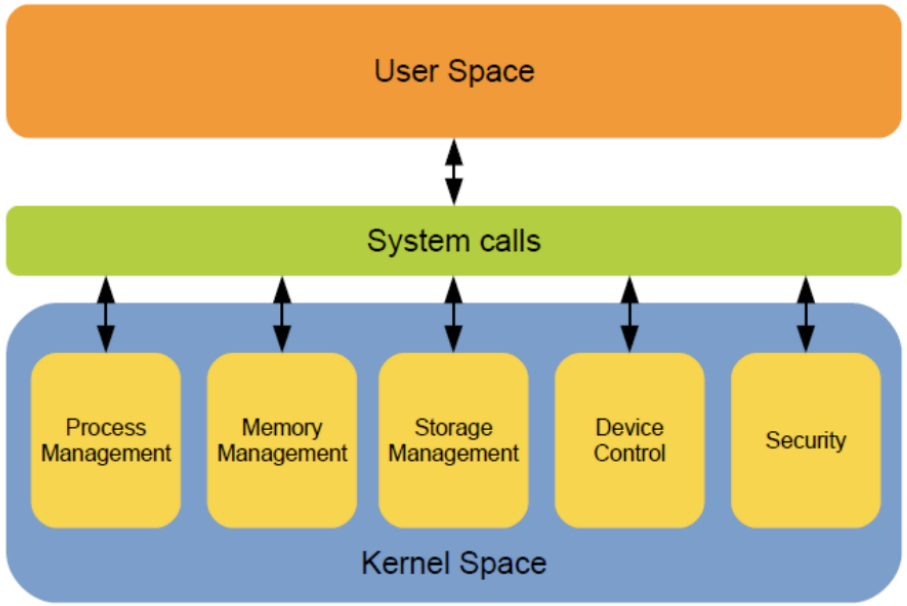
Ivy Peng

Associate Professor

EECS, KTH

OS Organization

- Process management:
  - Creating and deleting processes.
  - Scheduling processes and threads on the CPUs.
  - Suspending and resuming processes.
  - Process synchronization.
  - Inter-process communication.





Outline

- **What is a Process**
- Processes Operations
- Process Scheduling
- Inter-process Communication

What is a process?

A program is a passive entity stored on a disk (executable file), while a process is an active entity.

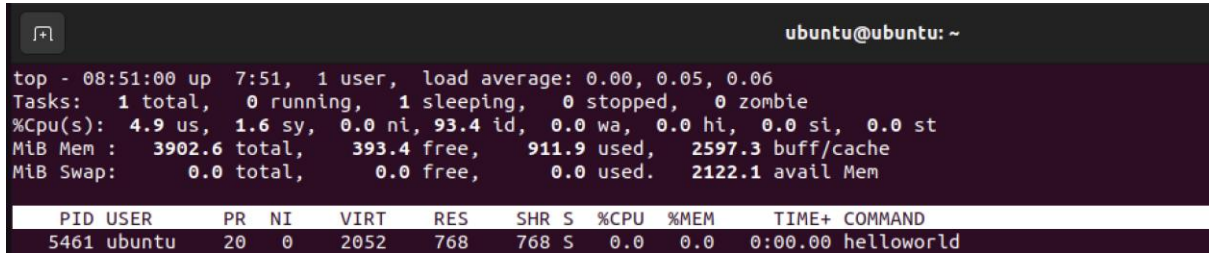
The program becomes a process when an executable file is loaded into memory.

One program can be several processes.

```
#include <stdio.h>

void main() {
  printf("Hello World!\n");
}
```

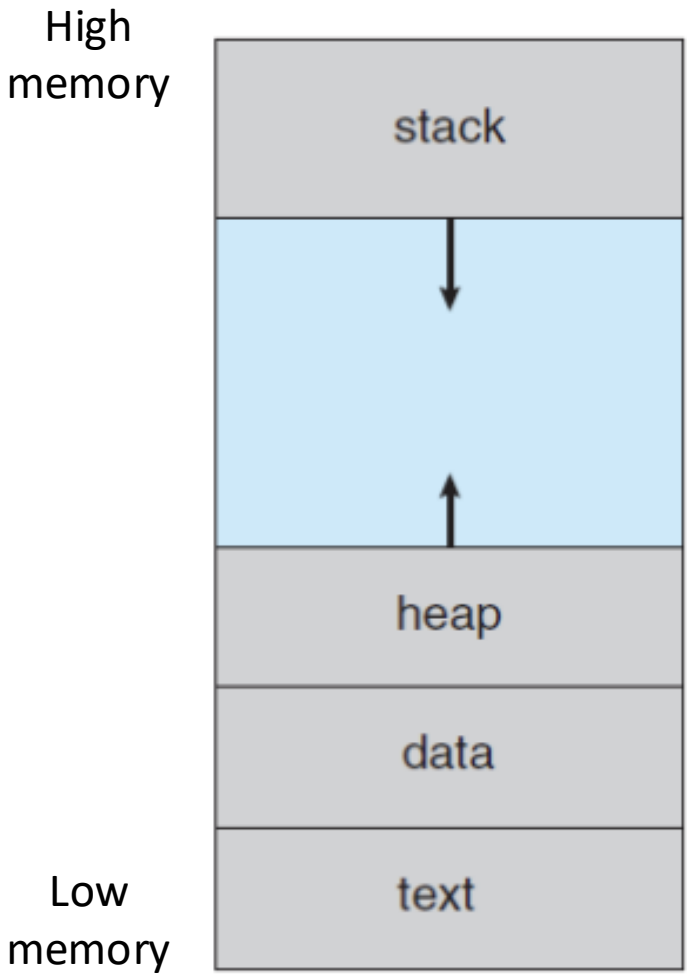
program



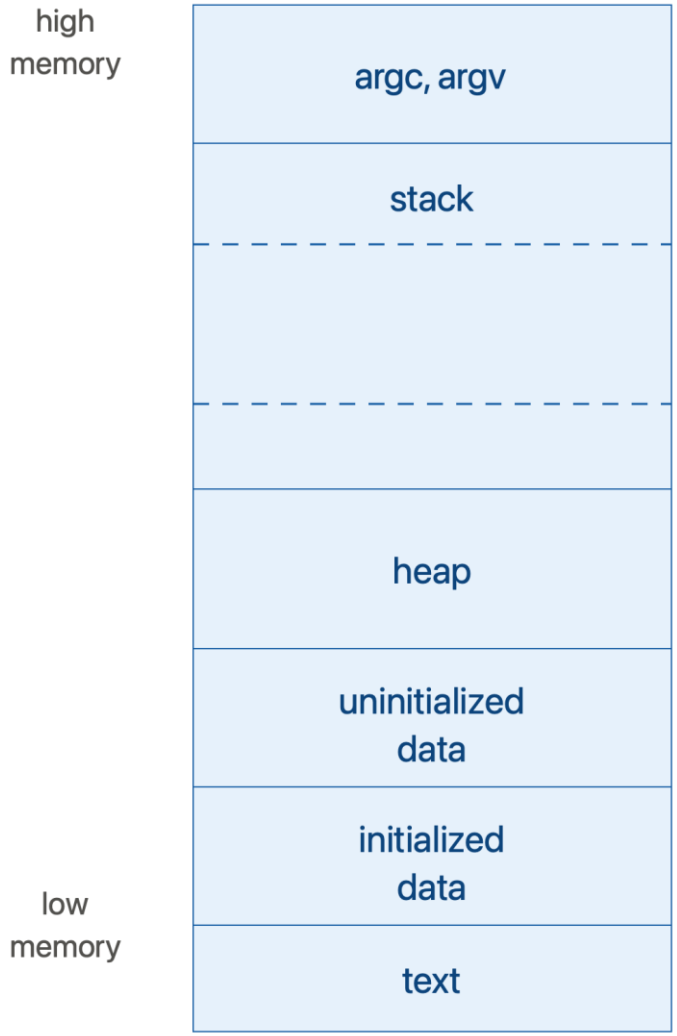
process

Memory Layout of a Process

- Four main parts of a process in memory:
  - **Text section**: the executable code
  - **Data section**: global variables
  - **Heap section**: dynamically allocated memory at run time
  - **Stack section**: temporary data storage when invoking functions
- Stack overflow



Memory Layout of a Process




```c
#include <stdio.h>
#include <stdlib.h>

int x;
int y = 15;

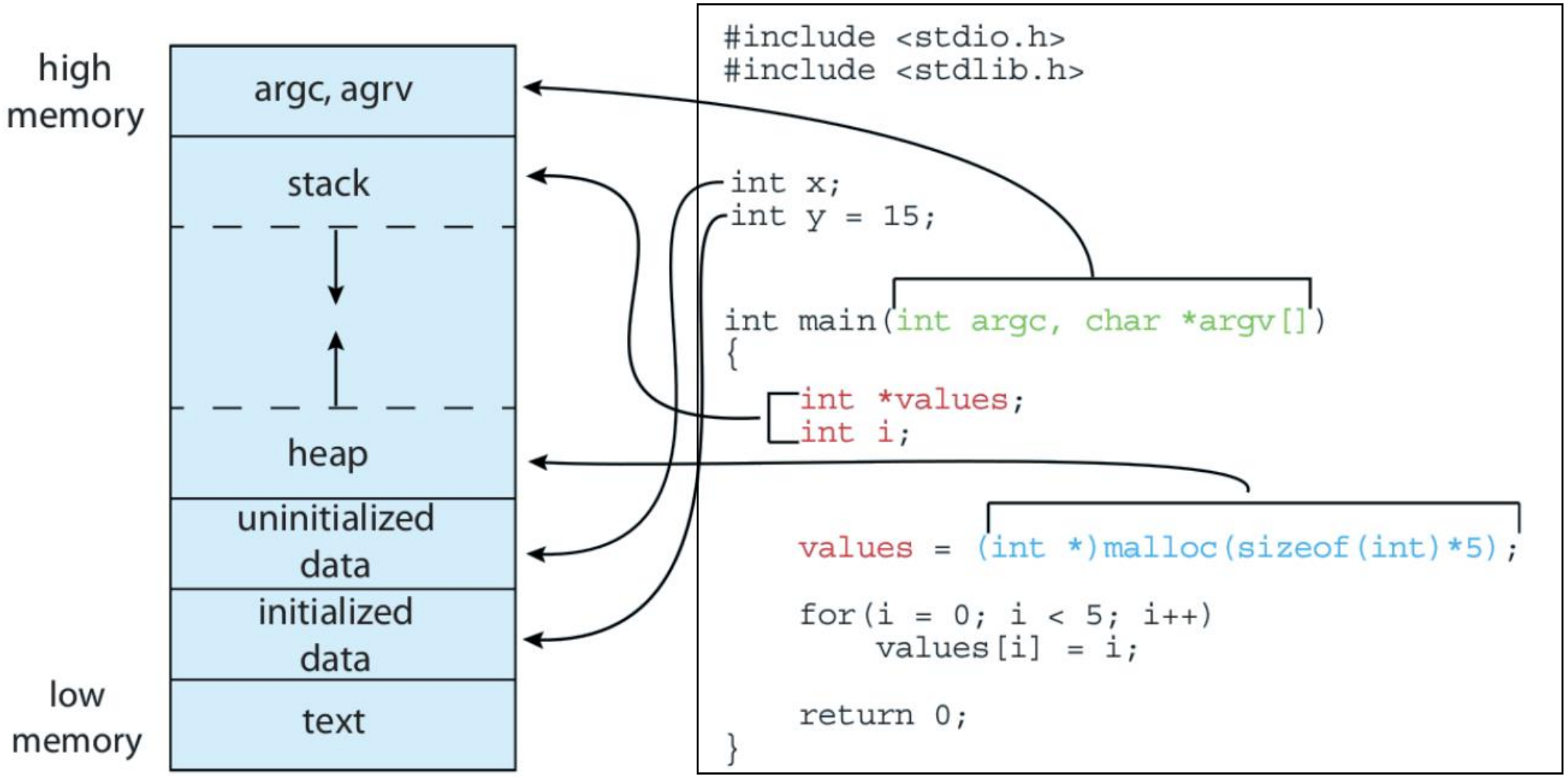
int main(int argc, char *argv[])
{
   int *values;
   int i;

   values = (int *)malloc(sizeof(int)*5);

   for(i = 0; i < 5; i++)
      values[i] = i;

   return 0;
}
```

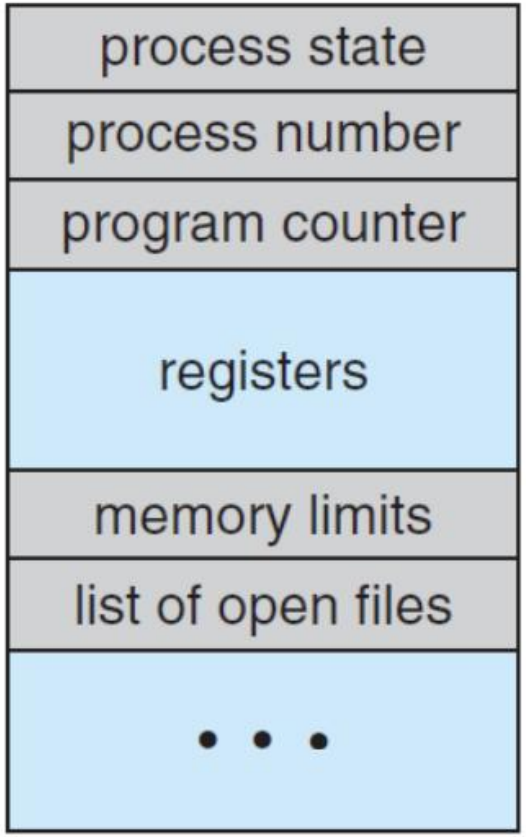
Memory Layout of a Process



Process Control Block

- OS needs to manage all processes on a system
- Each process is represented in a Process Control Block (PCB)
- In the Linux kernel, PCB is implemented by the task struct.
  - At <include/linux/sched.h>, HW 1

Process Control Block (PCB)



More about task_struct in Linux

- Linux kernel implements PCB in task_struct at `<include/linux/sched.h>`
  - cpu info
  - State
  - Pid, parent, child, …
  - Scheduling info

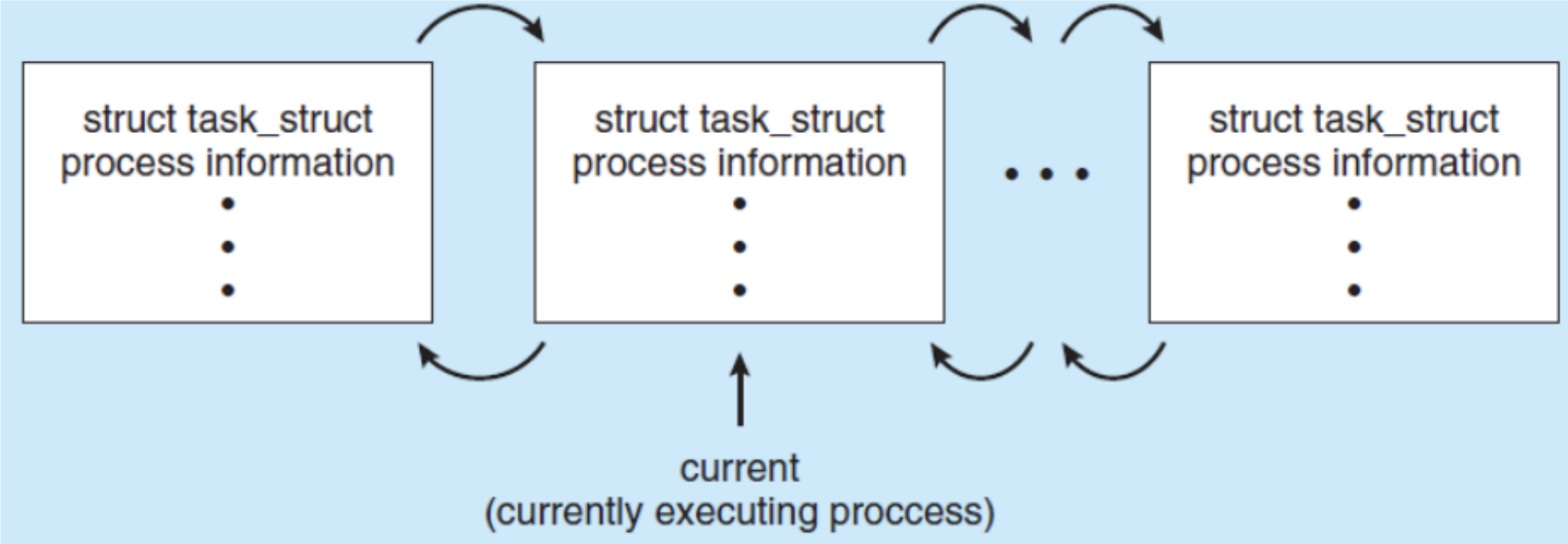
- Example: https://elixir.bootlin.com/linux/v6.12.41/source/include/linux/sched.h

include / linux / sched.h

```
785  struct task_struct {
786  #ifdef CONFIG_THREAD_INFO_IN_TASK
787          /*
788           * For reasons of header soup (see current_thread_info()), this
789           * must be the first element of task_struct.
790           */
791          struct thread_info              thread_info;
792  #endif
793          unsigned int                    __state;
794
795          /* saved state for "spinlock sleepers" */
796          unsigned int                    saved_state;
797
798          /*
799           * This begins the randomizable portion of task_struct. Only
800           * scheduling-critical items should be added above here.
801           */
802          randomized_struct_fields_start
803
804          void                            *stack;
805          refcount_t                      usage;
806          /* Per task flags (PF_*), defined further below: */
807          unsigned int                    flags;
808          unsigned int                    ptrace;
809
810  #ifdef CONFIG_MEM_ALLOC_PROFILING
811          struct alloc_tag                *alloc_tag;
812  #endif
813
814  #ifdef CONFIG_SMP
815          int                             on_cpu;
816          struct __call_single_node       wake_entry;
817          unsigned int                    wakee_flips;
818          unsigned long                   wakee_flip_decay_ts;
819          struct task_struct              *last_wakee;
820
821          /*
822           * recent_used_cpu is initially set as the last CPU used by a task
823           * that wakes affine another task. Waker/wakee relationships can
824           * push tasks around a CPU where each wakeup moves to the next one.
825           * Tracking a recently used CPU allows a quick search for a recently
826           * used CPU that may be idle.
827           */
828          int                             recent_used_cpu;
829          int                             wake_cpu;
830  #endif
831          int                             on_rq;
832
833          int                             prio;
834          int                             static_prio;
835          int                             normal_prio;
```

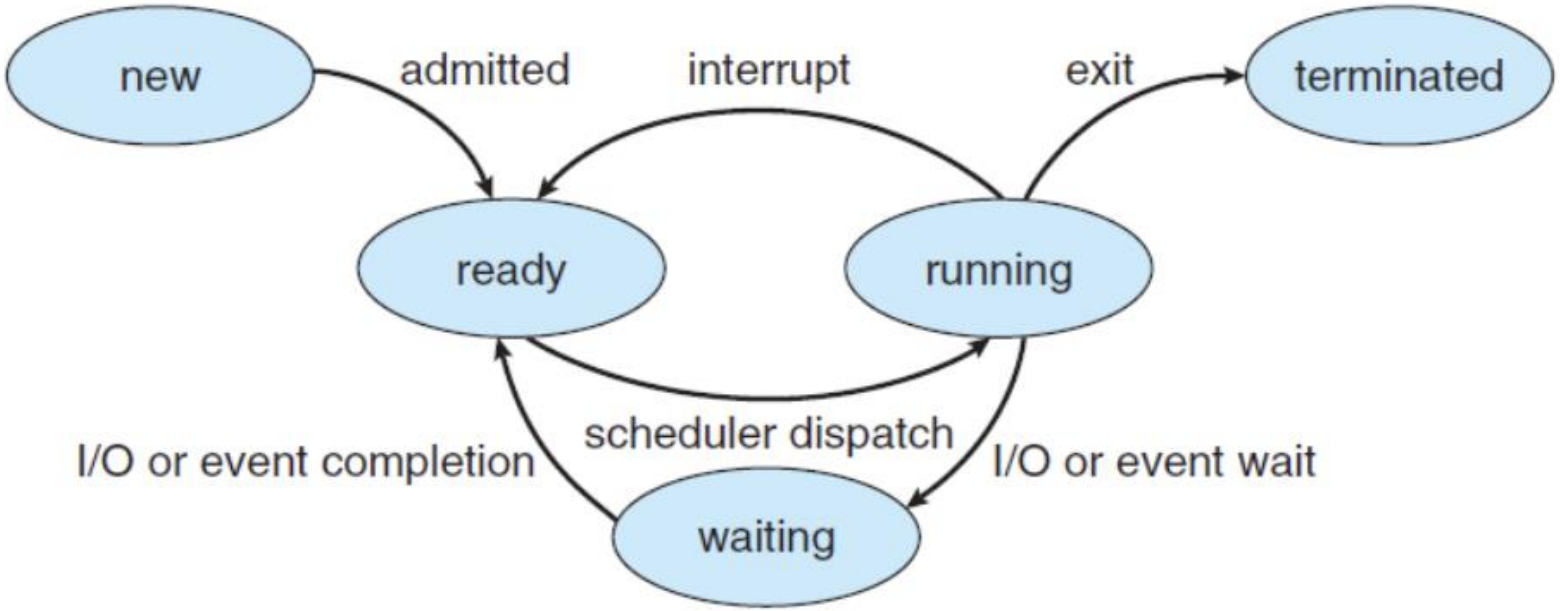
More about task_struct in Linux

- All active processes are represented using a circular doubly linked list of task_struct -> task list



Process State

- As a process executes, it changes state.
  - **New**: The process is being created.
  - **Ready**: The process is waiting to be assigned to a processor.
  - **Running**: Instructions are being executed.
  - **Waiting:** The process waits for an event to occur.
  - **Terminated**: The process has finished execution.



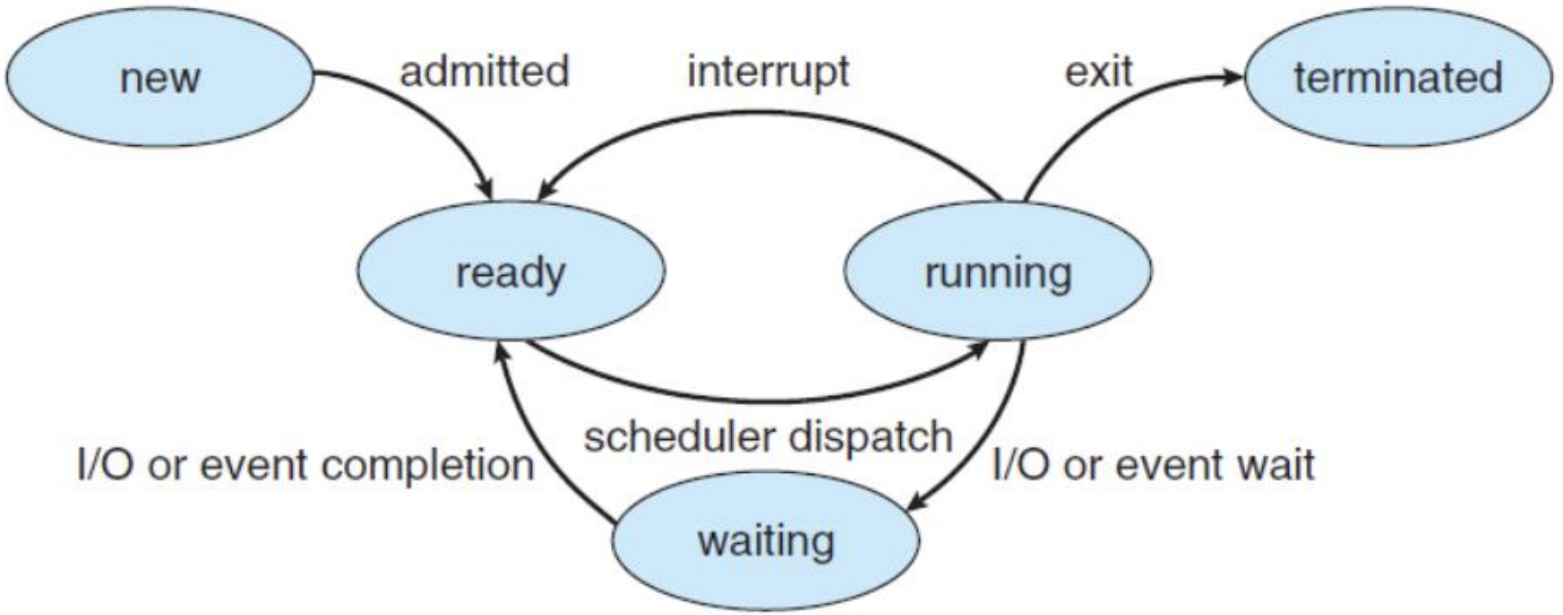


Outline

- What is a Process
- **Processes Operations**
- Process Scheduling
- Inter-process Communication

A Lifecycle of Process

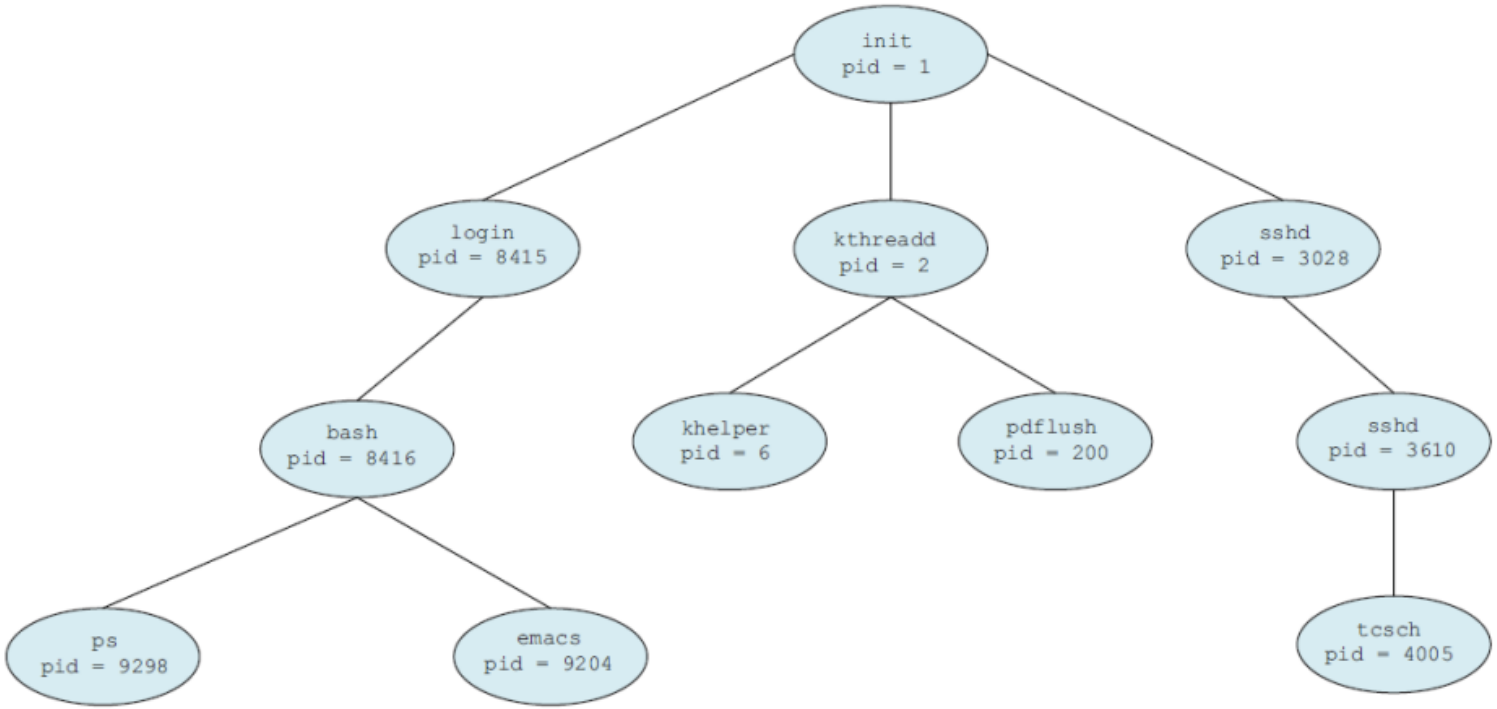
- Process creation: allocate needed computing resources.
- Process scheduling: when a process gets the CPU to execute
- Process termination: reclaim those computing resources.
- OS provides system calls for these process manipulation processes



Process Creation

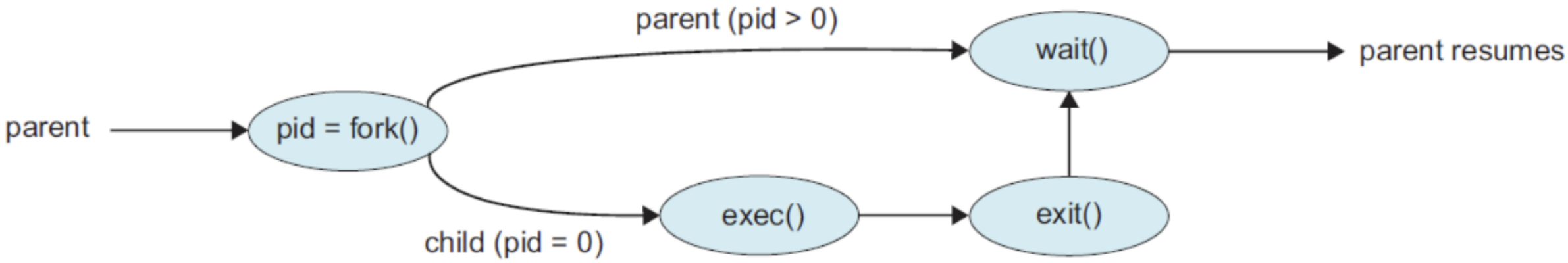
- A process may create several new processes.
  - The creating process: the parent
  - The new processes: the children

- These processes form a **tree of processes**.
  - Parent
  - Child
  - Sibling



Process Creation – fork( )

- fork() creates a new process.
  - The child process receives a logical copy of the parent's address space: page table copied, but pages are not, CoW.
  - The child continues execution at the instruction after the fork().
  - Use exec() to replace its memory image with a new program
- Each process is assigned a unique identifier, the process ID (PID).
  - The getpid() system call returns the PID of the calling process.



fork(): Called Once, Returns Twice

```
printf("before fork: pid=%d\n", getpid());

pid_t pid = fork();
if (pid < 0) { perror("fork"); exit(1); }

if (pid == 0)
{   printf("child : pid=%d ppid=%d, fork returned %d\n",
           getpid(), getppid(), pid);
} else {
    printf("parent: pid=%d, fork returned %d\n", getpid(), pid);
    wait(NULL);
}
printf("both reach here (pid=%d)\n", getpid());
```

One example output

```
1. before fork: pid=4210
2. parent: pid=4210, fork returned 4211
3. child : pid=4211 ppid=4210, fork returned 0
4. both reach here (pid=4211)
5. both reach here (pid=4210)
```

Which order is nondeterministic?

Called once by the parent process

Returned twice: in parent and child processes because the child inherit the parent's memory

Process Creation – `clone()`

- Similar to fork(), but more control of what is shared between parent and child
- Separate stack memory space
- The child process commences execution from the function fn.

```
int clone(typeof(int (void *_Nullable)) *fn,
          void *stack,
          int flags,
          void *_Nullable arg, ...
          /* pid_t *_Nullable parent_tid,
             void *_Nullable tls,
             pid_t *_Nullable child_tid */ );
```

The exec() Family

- The exec() family of functions replaces the current process image with a new process image to **execute a new program**.

- Used after fork() to replace the process memory space with a new program.

```c
#include <unistd.h>
int execl(const char *path, const char *arg, ...);
```

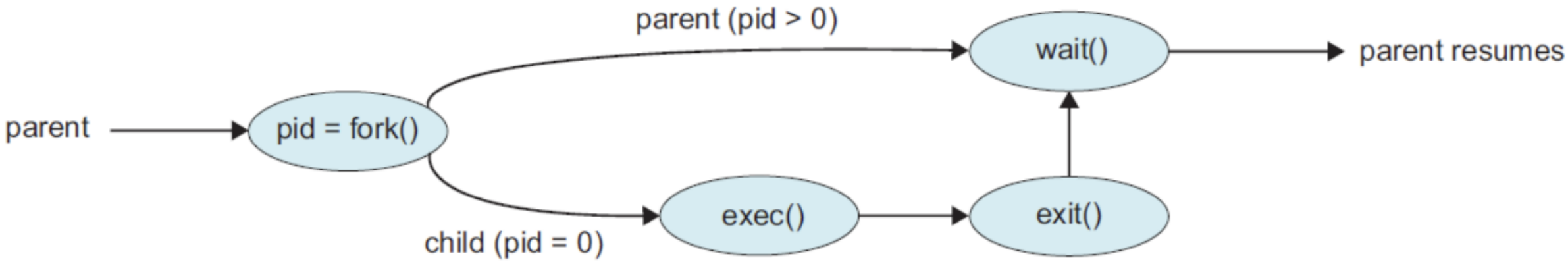
```c
pid_t pid = fork();

if (pid == -1) {
  perror("fork");
  exit(1);
}

if (pid == 0) { // the child
  const char *args[] = {"windlass", NULL};
  int ret;
  ret = execv("/bin/windlass", args);
  if (ret == -1) {
    perror("execv");
    exit(1);
  }
}
```

Process Termination

- Use **exit()** to instruct the OS to terminate the process.
  - Returns status data from the child to the parent.
  - The OS deallocates/releases process resources.
- The parent process may wait for the termination of a child via wait() or waitpid().
  - If a process has terminated but its parent has not yet called wait(), the process is a **zombie**.
  - If the parent terminated without invoking wait(), the process is an **orphan**.



Put Things All Together

- Demo: 2-demo-process-wait.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_CHILDREN 3

int main() {
    pid_t pids[NUM_CHILDREN];

    for (int i = 0; i < NUM_CHILDREN; i++) {  // Create multiple child processes
      pid_t pid = fork();

      if (pid < 0) {
        perror("fork failed");
        exit(1);
      }
      else if (pid == 0) {// Child process block

        // Make children exit in an unpredictable order
        unsigned int sleep_time = (NUM_CHILDREN - i);  // later children exit sooner
        printf("Child %d started: PID = %d, Parent PID = %d, will sleep %us\n",
               i, getpid(), getppid(), sleep_time);
        sleep(sleep_time);

        printf("Child %d exiting with code %d ...\n", i, 8 + i);
        exit(8 + i);  // unique exit status per child
      }
      else {// Parent process block
        pids[i] = pid;
      }
    }

    printf("\nParent process: PID = %d, waiting for %d children ...\n\n",
           getpid(), NUM_CHILDREN);

    int child_status;
    pid_t ret_child_pid;

    while ((ret_child_pid = wait(&child_status)) > 0) {
      printf("Reaped child PID = %d, exit status = %d\n",
             ret_child_pid, WEXITSTATUS(child_status));
    }

    printf("\nParent process: all children reaped. Exiting.\n");
    return 0;
}
```

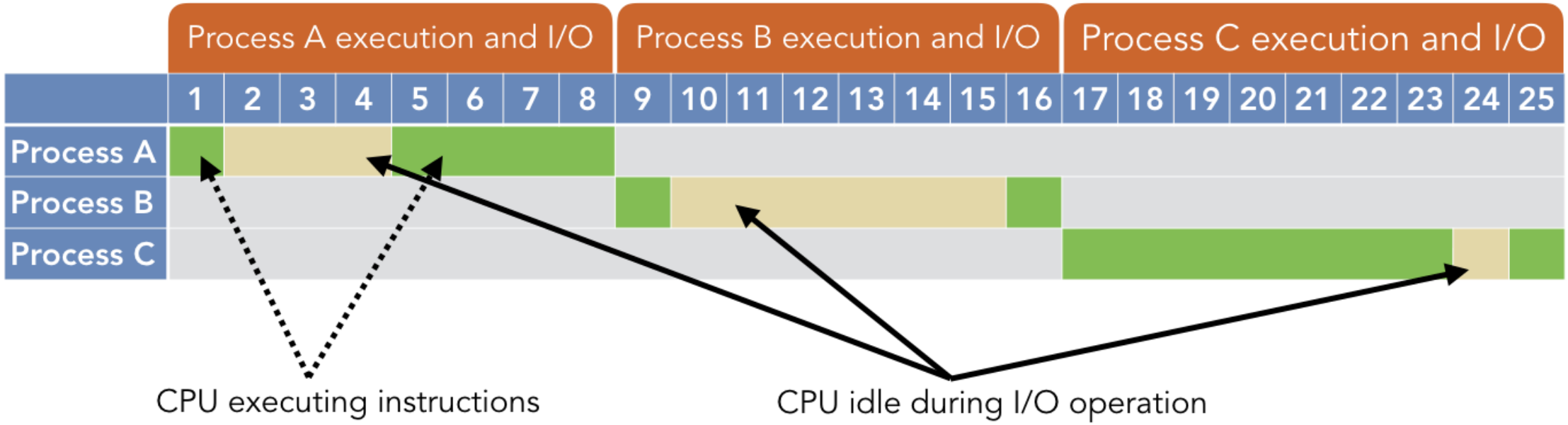


Outline

- What is a Process
- Processes Operations
- **Process Scheduling**
- Inter-process Communication

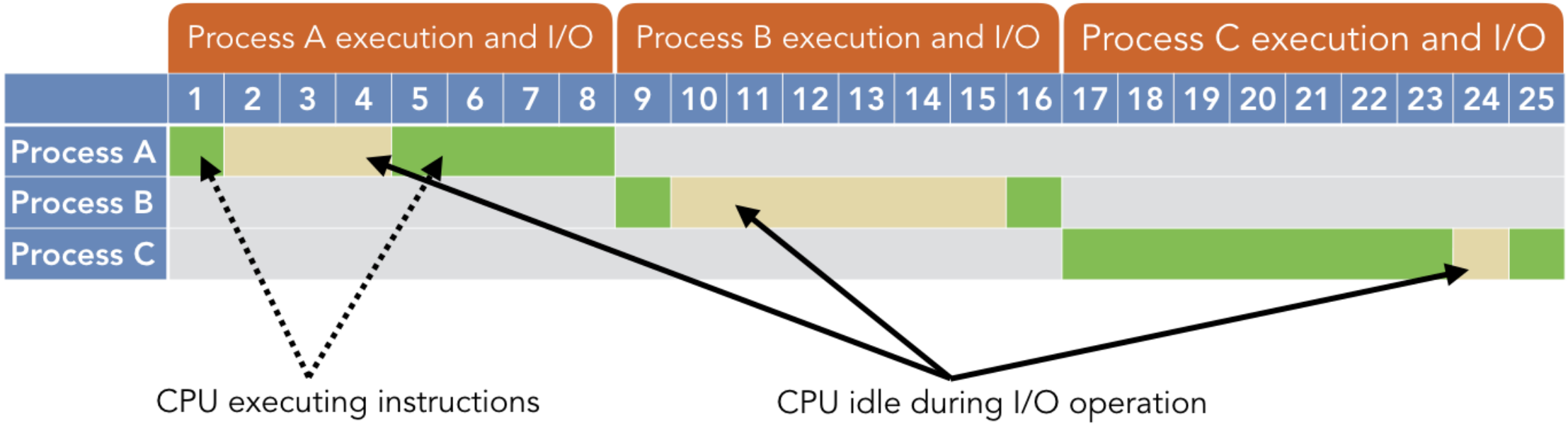
Process Scheduling

- One CPU core can only run a process at a time
  - single core system
- If there are more processes than cores
  - Process will have to wait until a core is free and can be rescheduled



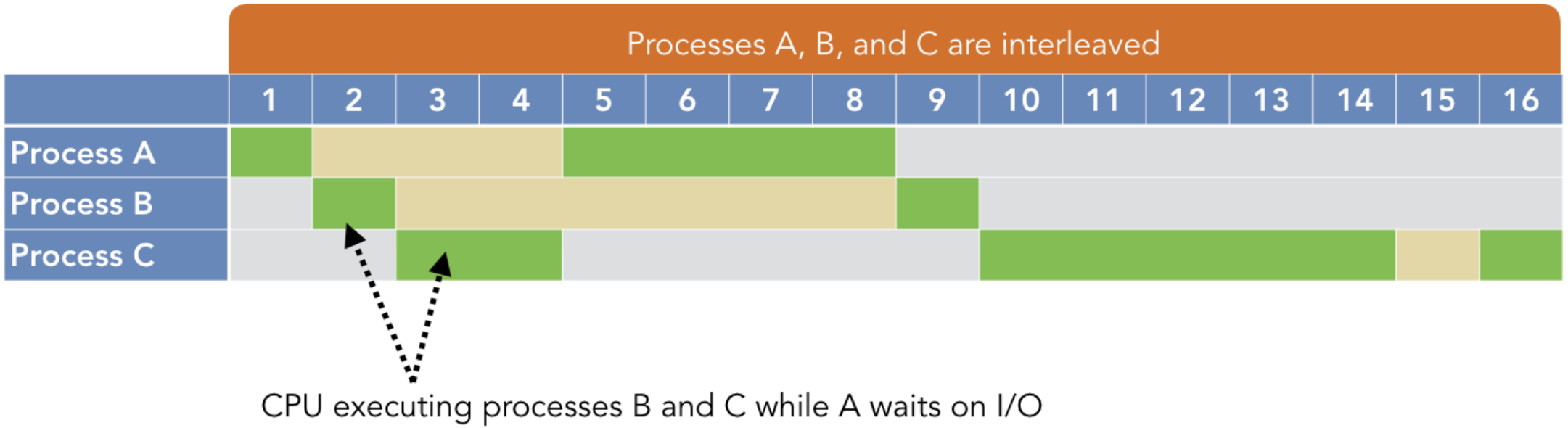
CPU Utilization

- CPU Util (%) = $\frac{Time\ that\ CPU\ was\ used}{Total\ Time}$ = $\frac{15}{25}$ = 60%



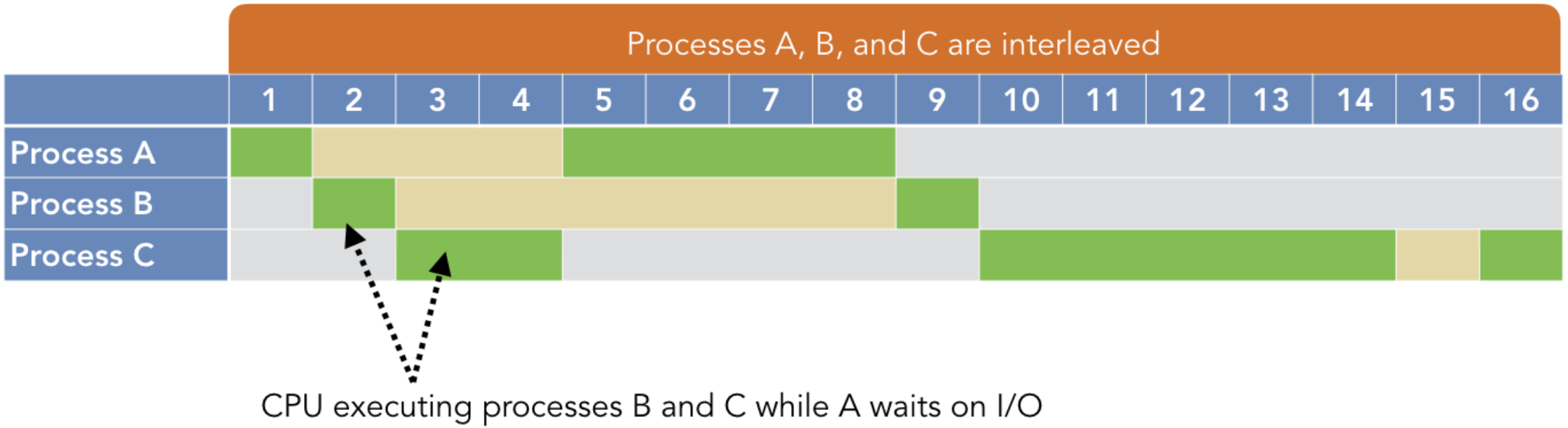
Multiprogramming

- the CPU is idle while waiting for very slow I/O operations
- **Multiprogramming** is used to reduce this wasted CPU time
  - Multiprogramming loads multiple processes into memory
  - When one process is executing an I/O operation, the kernel switches to execute another process



CPU Utilization

- CPU Util (%) = $\frac{Time\ that\ CPU\ was\ used}{Total\ Time} = 15/16 = 93.7\%$



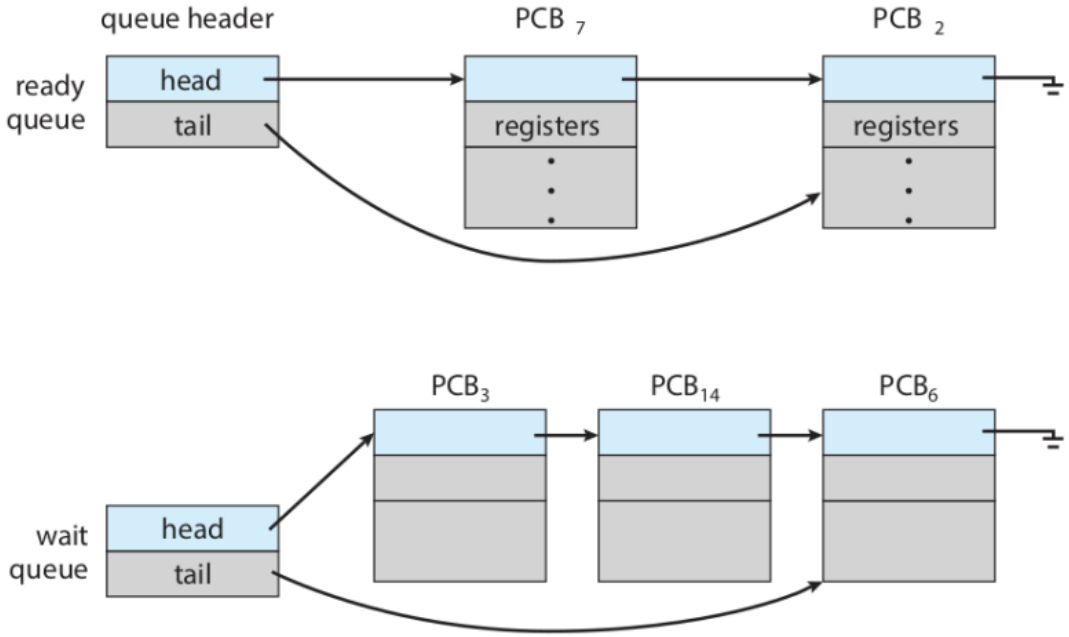
Multiprogramming and Scheduling

How to decide when to switch to another process? For how long?

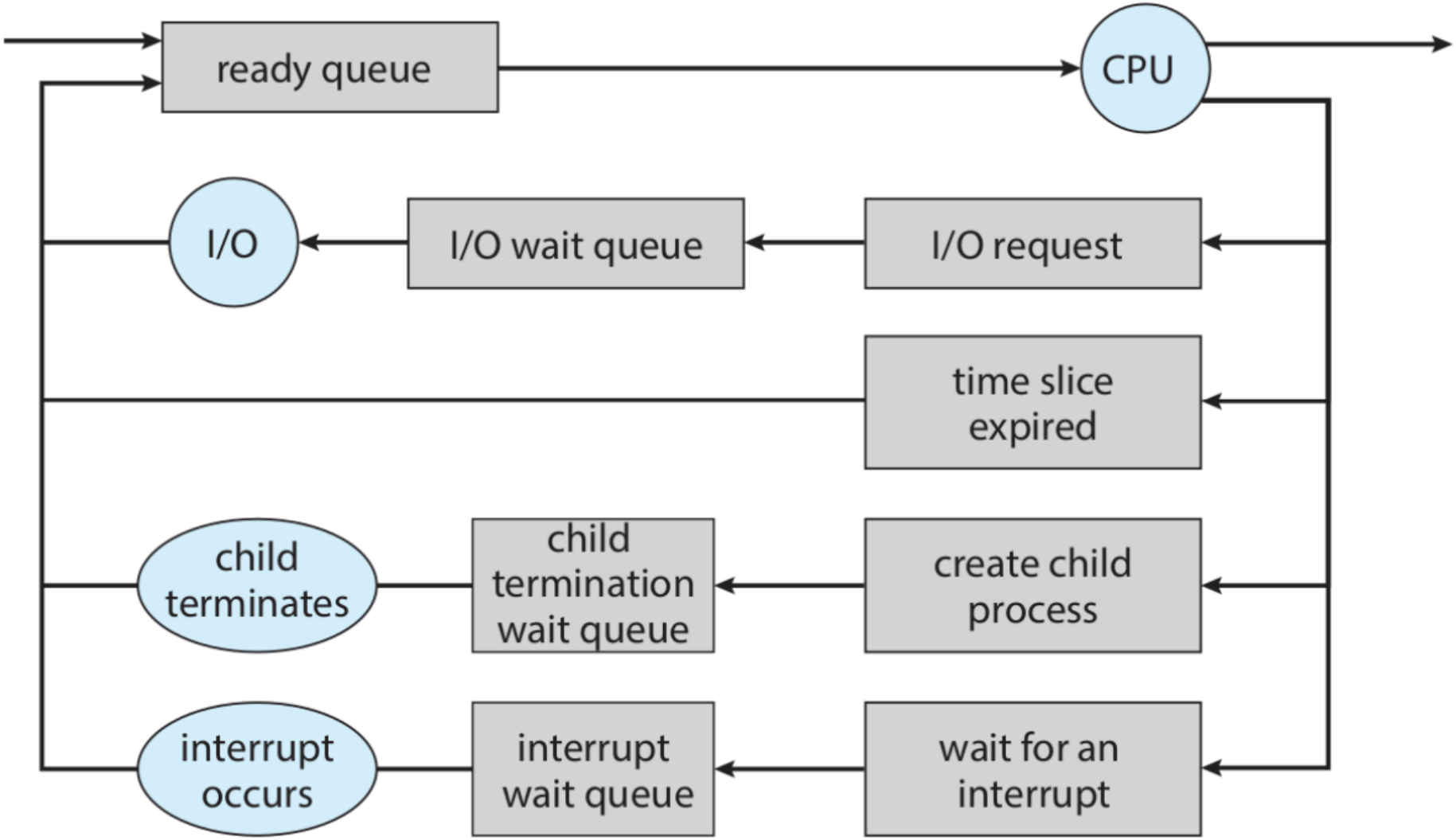
- **preemptive** multitasking
  - A time limit
- **cooperative** multitasking
  - voluntarily relinquishes control
- Pros & Cons?
- We will see more in the CPU scheduling lecture

Scheduling Queues

- **Ready queue**: a set of all processes residing in main memory, in ready state, waiting to execute.
- **Wait queues**: a set of processes waiting for an even (e.g., a response from I/O device).

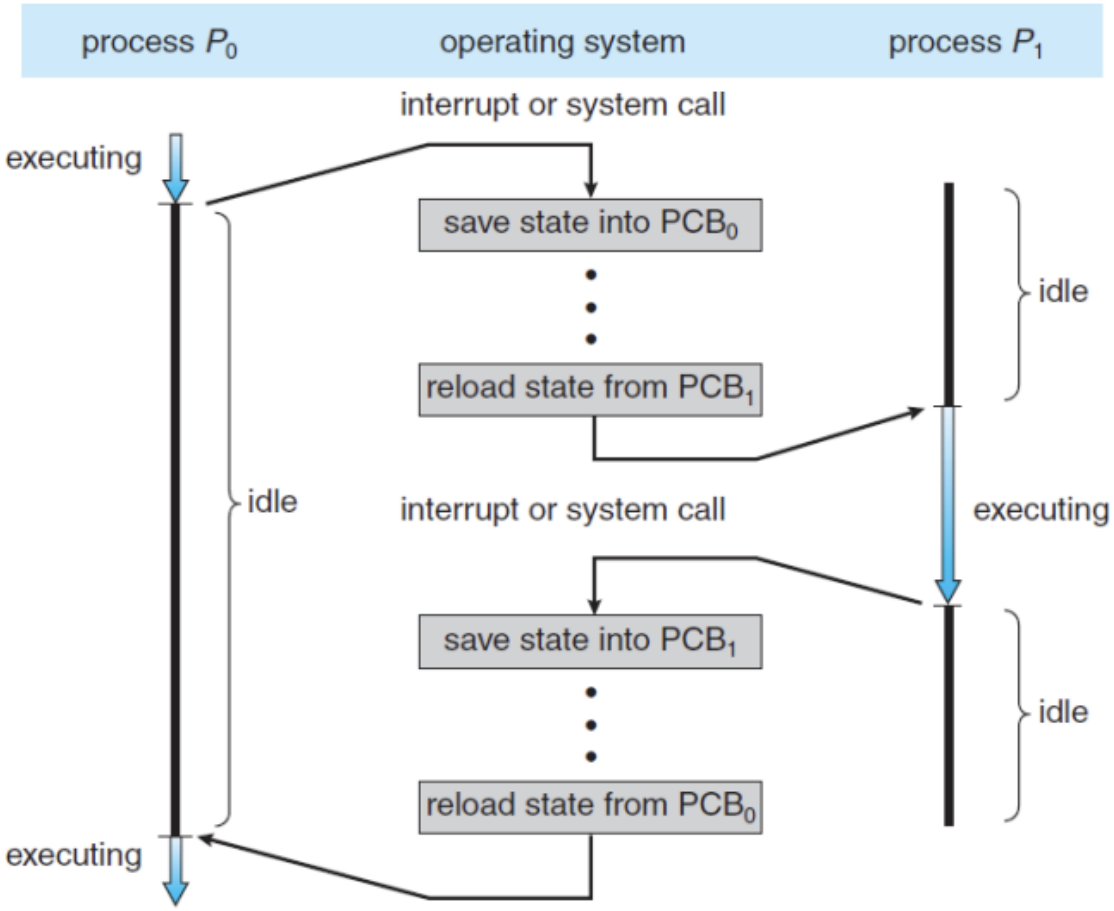


Queueing Diagram



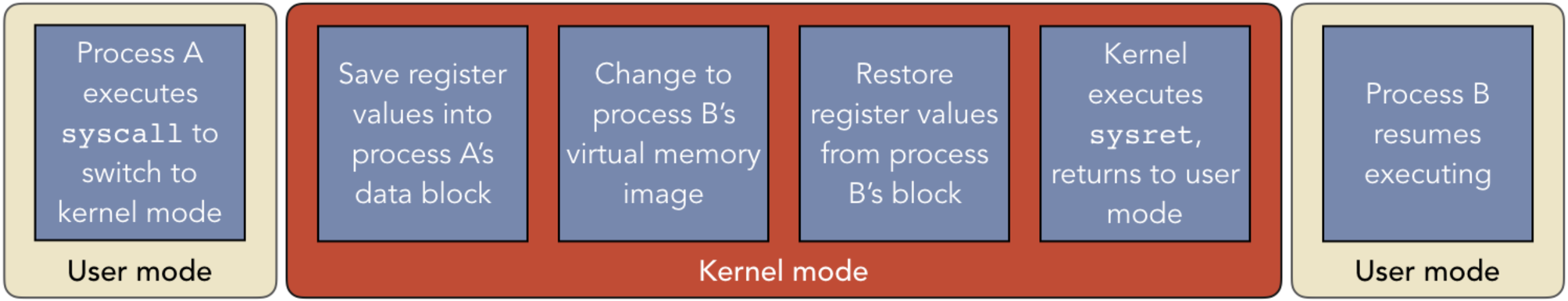
Context Switching

- It is the basis of multiprogramming
- When the CPU switches to another process:
  - The system saves the state of the old process.
  - The saved state of the new process is loaded via a context switch.
  - It's called **context switching**.
- The context of a process is represented in the PCB.



The Cost of Context Switching

- Context switch runs a scheduling algorithm to make decision on what task to run
- changes from one process's virtual memory image to another.
  - data, heap, and stack segments that the CPU uses are changed
  - Cache, memory, data access



[https://w3.cs.jmu.edu/kirkpams/OpenCSF/Books/csf/html/Multiprogramming.html#context-switches-and-overhead-costs]

time

Switch the execution from Process A -> Process B

Reduce the Cost of Context Switching

- A context switch saves the CPU registers of the process being switched out and restores the registers of the process being scheduled in.

- The direct cost: registers plus the scheduler, is microseconds. The indirect cost is usually larger: the incoming process resumes with a cold cache and TLB. Need to which cost dominates.

- Reduce the time spent context switching by:
  - Hardware support for saving and restoring processor state.
  - Increase the interval between switches (a longer scheduling quantum)

HW1 will need you to measure the cost

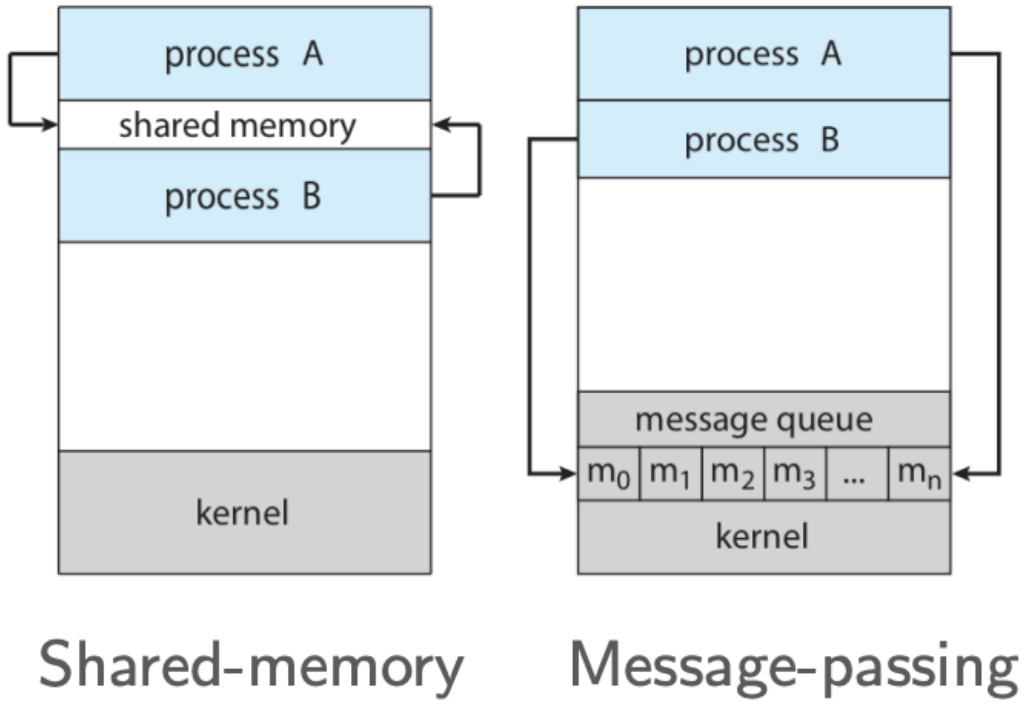


Outline

- What is a Process
- Processes Operations
- Process Scheduling
- **Inter-process Communication**

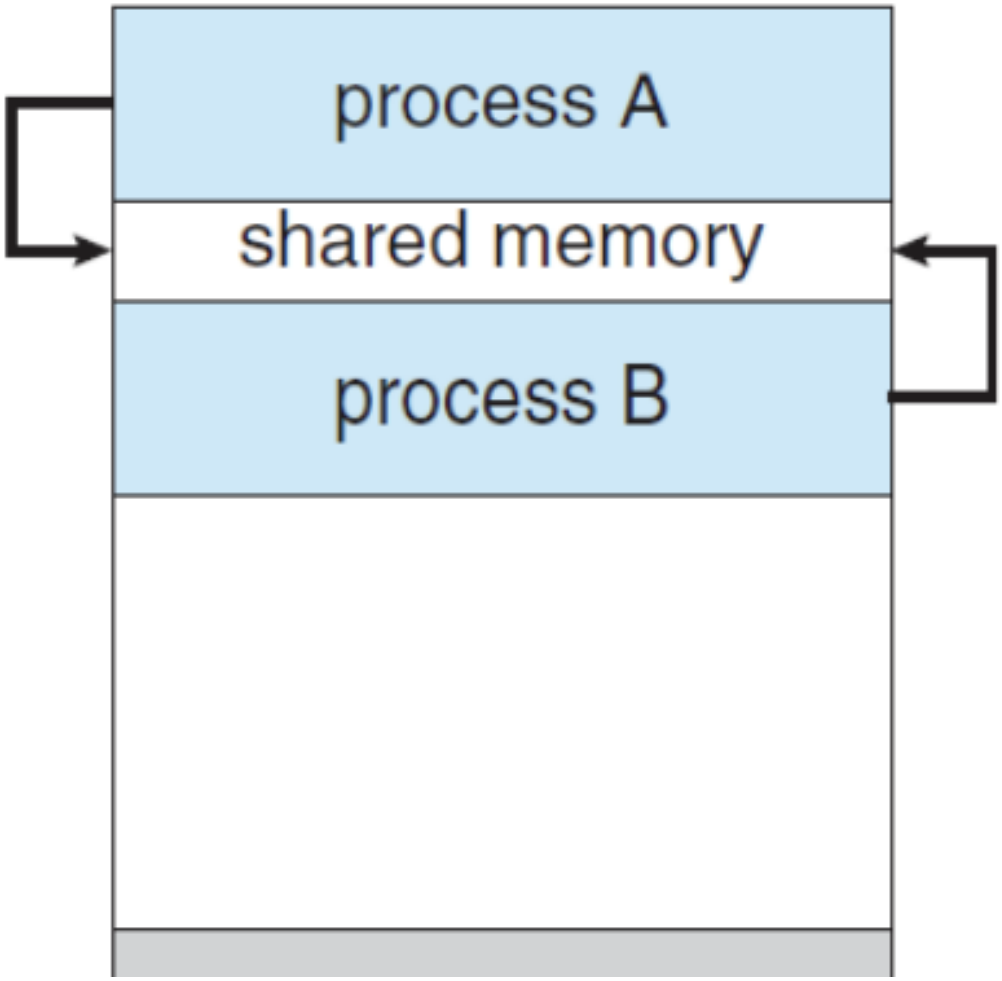
Inter-Process Communication (IPC)

- IPC mechanisms allow processes to exchange data.
- Two models of IPC
  - Shared memory
  - Message Passing
    - Data stream:
      - Pipe
      - FIFO (Named Pipe)
    - Message queue



IPC - Shared Memory (1/2)

- An area of memory shared among the processes that wish to communicate.
- It resides in the process's address space, creating the shared-memory segment.
- Isn’t process isolation part of OS’s job?
  - Access Control : require the processes to remove this restriction



IPC - Shared Memory (2/2)

- `shm_open()` creates and opens a new shared memory object or an existing one.
- `mmap()` creates a new mapping in the virtual address space of the calling process.
- `shm_unlink()` removes a shared memory object.

Producer

```c
int SIZE = 4096;
char *my_shm = "/tmp/myshm";
char *write_msg = "hello";
char *addr;
int fd;

// create the shared memory object
fd = shm_open(my_shm, O_CREATE | O_RDWR, 0666);

// configuare the size of the shared memory object
ftruncate(fd, SIZE);

// memory map to the shared memory object
addr = mmap(NULL, SIZE, PROT_WRITE, MAP_SHARED, fd, 0);

// write to the shared object
sprintf(addr, "%s", write_msg);
```

Consumer

```c
int SIZE = 4096;
char *my_shm = "/tmp/myshm";
char *addr;
int fd;

// open the shared memory object
fd = shm_open(my_shm, O_RDONLY, 0666);

// memory map to the shared memory object
addr = mmap(NULL, SIZE, PROT_READ, MAP_SHARED, fd, 0);

// read from to the shared object
printf("%s", (char *)addr);

// remove the shared memory object
shm_unlink("my_shm");
```

IPC - Pipe (1/3)

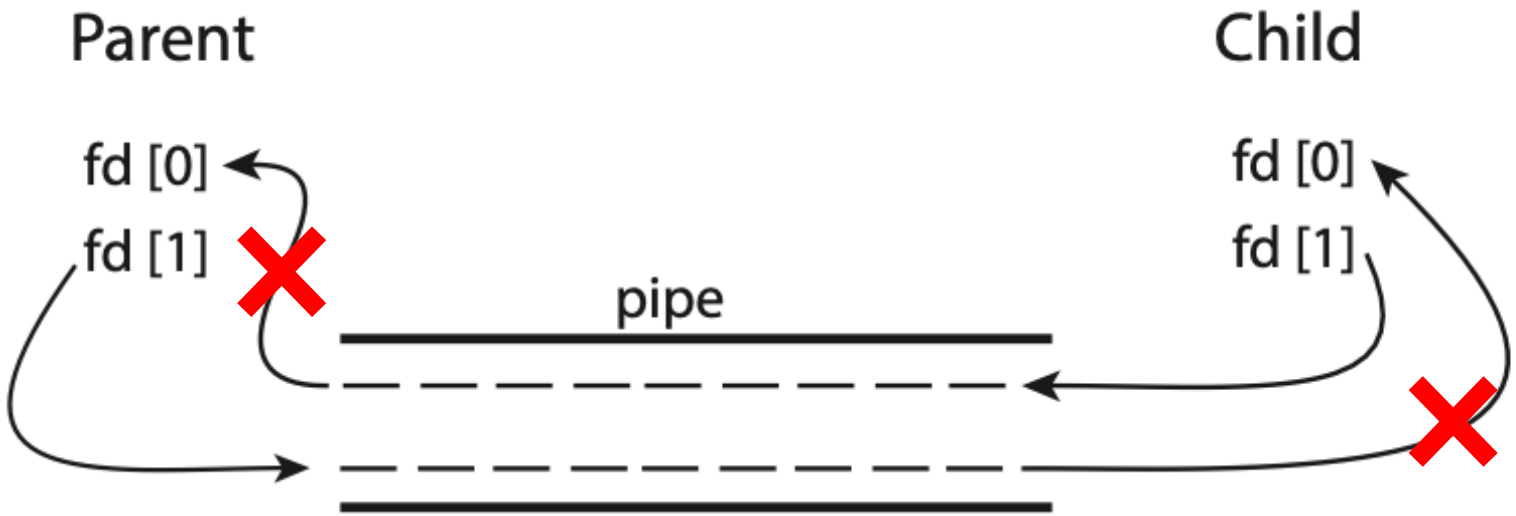
- Also called ordinary pipe
- Allow **two** processes to communicate in a producer– consumer fashion
  - Pipes are unidirectional, allowing only one-way communication.
  - Require parent-child relationship between communicating processes.
  - only for communication between processes on the same machine

Pipe in Practice: in the UNIX, the output of one command serves as input to another.



IPC - Pipe (2/3)

- On UNIX systems, a pipe is a special type of file: can be accessed using ordinary read() and write() system calls.
- constructed using the function pipe(int fd[]), which returns two open file descriptors in fd[2]:
  - fd[0] to read from the pipe, fd[1] to write to the pipe
  - Both the parent process and the child process need to close their unused ends of the pipe, close(fd[READ_END])



[Operating System Concepts, 3.7.4.1]

IPC – Pipe (3/3)

- Demo

```
int fd[2];
if (pipe(fd) == -1) { perror("pipe"); return 1; }

/* fd[0] = read end,  fd[1] = write end */

if (fork() == 0) {                  /* CHILD = producer  */
    close(fd[0]);                   /* unused READ end   */
    char *m = "hello from the child\n";
    write(fd[1], m, strlen(m));
    close(fd[1]);                   /* -> EOF for reader */
    _exit(0);
}

close(fd[1]);                       /* PARENT = consumer */
char buf[64]; ssize_t n;
while ((n = read(fd[0], buf, sizeof buf)) > 0)   /* 0 = EOF */
    write(STDOUT_FILENO, buf, n);
close(fd[0]);
wait(NULL);
```

IPC - Named Pipe/FIFO (1/2)

- Communication can be bidirectional (need two buffers). Multiple processes can communicate.

```
#include <sys/stat.h>
int mkfifo(const char *pathname, mode_t mode);
```

- No parent-child relationship is required
  - Multiple processes can communicate
  - Continue to exist until it is explicitly deleted
- In UNIX, named pipes are called FIFOs
  - created with the mkfifo() system call
- Treated as typical files in the file system
  - open(), read(), write(), and close() system calls

IPC - Named Pipe/FIFO (2/2)

- Implicit synchronization in two distinct forms:
  - open() blocks until the FIFO is open at both ends (see slide 16).
  - write() blocks only when the buffer is full (64 KiB by default); read() blocks when it is empty.

Producer

```c
#include <sys/stat.h>
#include <sys/types.h>
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

//int mkfifo(const char *pathname, mode_t mode);

char *my_fifo = "myfifo";
char *write_msg = "hello";
int fd;

int main(){
  // Create the FIFO (named pipe)
  mkfifo(my_fifo, 0666);

  // Write "hello" to the FIFO
  fd = open(my_fifo, O_WRONLY);
  write(fd, write_msg, strlen(write_msg));
  close(fd);

  // Remove the FIFO
  unlink(my_fifo);
}
```

Consumer

```c
#include <sys/stat.h>
#include <sys/types.h>
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define  MAX_SIZE 100


char *my_fifo = "myfifo";
char buf[MAX_SIZE];

int fd;

int main(){

  // Open the FIFO
  fd = open(my_fifo, O_RDONLY);

  // Read the message from the FIFO
  read(fd, buf, MAX_SIZE);
  printf("Received: %s\n", buf);

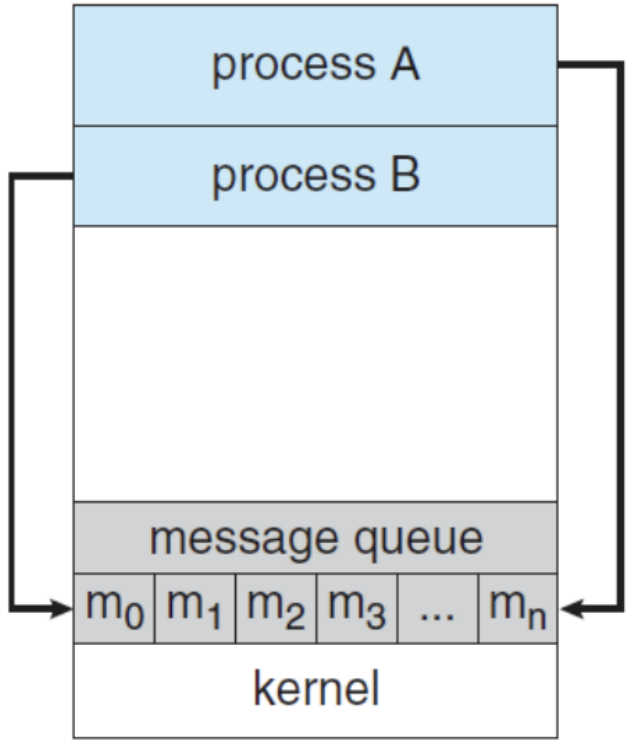
  // Close the FIFO
  close(fd);

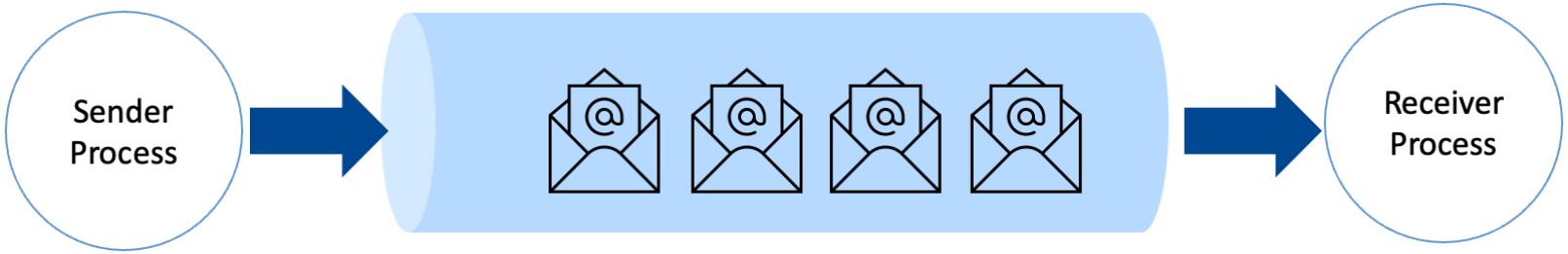
}
```

IPC - Message Passing

- Communicating with each other without resorting to shared variables as in shared memory.

- Data is structured: receive as a whole message
- Different Priority levels
- No synchronization: messages stay in the queue until picked up





POSIX Message Queue mqueue.h

- mqd_t mq_open (const char *name, int oflag, ... /* mode_t mode, struct mq_attr *attr */);
  - Open /create a POSIX message queue.
- int mq_close (mqd_t mqdes);
  - Close a message queue.
- int mq_unlink (const char *name);
  - Initiate deletion of a message queue.
- int mq_send (mqd_t mqdes, const char *msg_ptr, size_t msg_len, unsigned int msg_prio);
  - Send the message pointed to by msg_ptr with priority msg_prio.
- int mq_getattr(mqd_t mqdes, struct mq_attr *attr);
  - Get the attributes associated with a given message queue.
- ssize_t mq_receive (mqd_t mqdes, char *msg_ptr, size_t msg_len, unsigned int *msg_prio);
  - Receive a message into a buffer pointed to by msg_ptr and get its priority msg_prio.

POSIX Message Queue mqueue.h

- Sender

```c
#include <stdlib.h>
#include <stdio.h>
#include <mqueue.h>
#include <assert.h>

int main(){

  mqd_t mqd = mq_open ("/MQ", O_CREAT | O_EXCL | O_WRONLY,  0600, NULL);

  /* Ensure the creation was successful */
  if (mqd == -1)
    {
      perror ("mq_open");
      exit (1);
    }

  /* Send "HELLO" as a message with priority 10, then close the queue.
     Note the size is 6 to include the null byte '\0'. */
  mq_send (mqd, "HELLO", 6, 10);
  mq_close (mqd);
}
```

- Receiver

```c
#include <stdlib.h>
#include <stdio.h>
#include <mqueue.h>
#include <assert.h>

int main() {

  mqd_t mqd = mq_open ("/MQ", O_RDONLY);
  assert (mqd != -1);

  /* Get the message queue attributes */
  struct mq_attr attr;
  assert (mq_getattr (mqd, &attr) != -1);

  char *buffer = calloc (attr.mq_msgsize, 1);
  assert (buffer != NULL);

  /* Retrieve message from the queue and get its priority level */
  unsigned int priority = 0;
  if ((mq_receive (mqd, buffer, attr.mq_msgsize, &priority)) == -1)
    printf ("Failed to receive message\n");
  else
    printf ("Received [priority %u]: '%s'\n", priority, buffer);

  /* Clean up the allocated memory and message queue */
  free (buffer);
  buffer = NULL;
  mq_close (mqd);
}
```

Choose an IPC

	Shared memory	Pipe	FIFO	POSIX msg queue
Direction	bidirectional	one-way	one-way per FIFO	one-way per queue
Synchronization	none	implicit: blocks when full / empty	as pipe, plus open() blocks until both ends open	implicit; messages queue up
Data model	raw bytes random access	byte stream	byte stream	structured

Highest throughput



Summary

- Memory layout in a process
- Process states: new, running, waiting, ready, terminated
- Process scheduling: queues, context switching
- Inter-Process Communication